

חברת עבודה לניתוח תחבורה ציבורית בישראל עם Open Bus Stride API

החברת הזו נועדה להיות כלי עבודה מעשי: להבין את כל שכבות הנתונים של Open Bus, לזכור את המפתחות החשובים, לדעת מאיזה endpoint להתחיל, ולבנות ניתוחי תכנון-ביצוע אמיתיים. היא מבוססת על דף ה-Swagger של Open Bus Stride API, מפרט ה-OpenAPI החי, מאגרי GitHub של open-bus-stride-api, open-bus-stride-db, open-bus-pipelines, תיעוד משרד התחבורה ל-GTFS/SIRI, תקני GTFS הבין-לאומיים ומקורות משלימים כגון geo.mot.gov.il, data.gov.il, bus.gov.il.

תקציר מנהלים

Open Bus Stride הוא שכבת REST ציבורית שמנגישה נתוני תחבורה ציבורית בישראל כ-JSON, ומחברת בין GTFS כנתוני תכנון לבין SIRI כנתוני ביצוע בזמן אמת (Open Bus Stride API Docs). דף ה-API מפנה במפורש ל-STRIDE.md, למודל הנתונים של Stride DB, לתהליכי ה-ETL ולתיעוד מפתחים של משרד התחבורה, ולכן הדרך הנכונה ללמוד את המערכת היא לא endpoint בודד אלא שרשרת מלאה: מקור ממשלתי → API → DB → ETL (OpenAPI description) KPI.

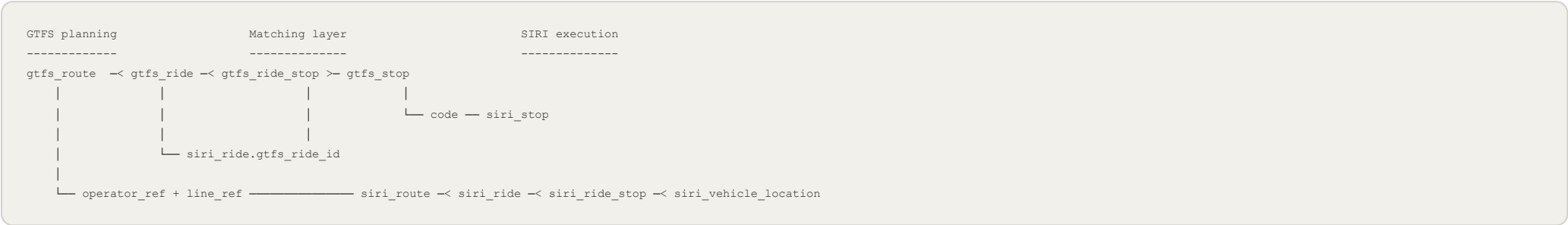
התמונה המרכזית היא הפרדה בין שני עולמות: gtfs_* מייצג את מה שתוכנן ליום שירות מסוים, ו-siri_* מייצג מה קרה בפועל לפי דגימות SIRI שנקלטות בתדירות גבוהה (Stride Data Model). כדי לחשב איחורים, ביצוע נסיעות, headway, משכי נסיעה או reliability, כמעט תמיד צריך לעבור דרך התאמה בין siri_ride ל-gtfs_ride ובין siri_ride_stop ל-gtfs_stop או (Stride DB repository) gtfs_ride_stop. ה-API כולל 27 endpoints בארבע משפחות: siri, gtfs, user cases, ו-(OpenAPI JSON) aggregations. בפועל, נקודת ההתחלה המומלצת לאנליסט היא user cases, כאשר רוצים תשובה מהירה, gtfs, כאשר מנתחים תכנון, siri, כאשר מנתחים ביצוע, ו-aggregations, כאשר רוצים ספירת מאקרו של מתוכנן מול מבצוע, אך בבדיקת שימוש חיה נמצא ש-gtfs_rides_agg/group_by עולול להחזיר שגיאת הרשאה למסד הנתונים ולכן יש להתייחס אליו בזהירות (בדיקת endpoint אגרסיבי).

מפת המקורות: מי מספק מה

מקור	מה הוא נותן	תדירות/רענונות	שימוש אנליטי	מגבלות
Open Bus Stride API	REST JSON מעל DB שמאחד GTFS ו-SIRI	SIRI כמעט בזמן אמת, GTFS יומי	שאלות מוכנות, ניתוח קווים, תחנות, ביצוע, מיקומים	שירות קהילתי, לא SLA ממשלתי, מגבלות limit ו-pagination (Open Bus Stride API Docs)
MOT GTFS	קווים, תחנות, נסיעות, stop_times, צורות ותעריפים	כל לילה	תכנון, היצע שירות, מרחקים, לוחות זמנים, שכבות GIS	יום שירות 28 שעות, route_id יומי, שדות ישראליים משתנים לאורך שנים (GTFS Documentation v3)
MOT SIRI-SM	חיזוי הגעה לתחנות ומיקומי כלי רכב	שניות עד דקה	ביצוע בפועל, ETA, מיקומי רכב, אמינות	דורש רישום ומפתח MOT במקור; Stride כבר קולט ומנרמל חלק מהמידע (SIRI-SM FAQ)
GTFS Realtime	TripUpdate, VehiclePosition, Alert לפי תקן Protobuf	תלוי feed	השוואה בין תקן בין-לאומי למימוש ישראלי	בישראל חלק ממקורות ה-realtime עדיין מתועדים דרך SIRI/ICD נפרד (GTFS Realtime Reference)
data.gov.il	מאגרי ממשלה נוספים, כולל רבי-קו/על-הקו ומאגרי תחבורה	משתנה לפי מאגר	ביקושים, שירות לקוחות, שכבות תומכות	לא כל מאגר מחובר ישירות ל-GTFS/SIRI; נדרש מיפוי מזהים (על-הקו)
geo.mot.gov.il	שכבות GIS, חיפוש קו/תחנה, buffer, heatmap, סטטיסטיקות	לפי שכבה	נגישות מרחבית, כיסוי, תשתיות, קירבה לתחנות	ממשק מפה ולא בהכרח API פתוח לכל שכבה (מערכת חצב)
KPI רשמי של הרשות	מסגרת מדדים: נסיעות נוסע, עלות, שביעות רצון, אמינות, זמינות	דוחות תקופתיים	מסגרת מושגית לבניית KPI משלך	חלק ממקורות המדד אינם פתוחים, למשל תיקופים מלאים (דוח מדדי ביצוע מרכזיים)

מודל חשיבה: תכנון מול ביצוע

הדרך לזכור את המערכת היא כציר של ארבעה שלבים: תכנון, ביצוע, התאמה, מדידה. GTFS עונה על "מה היה אמור לקרות"; SIRI עונה על "מה נראה שקרה בפועל"; Stride DB שומר את הקשרים ביניהם; וה-API מאפשר לשלוף את החתיכות לצורך (Stride Data Model) KPI.



חמש ישויות שחייבים לשנן

ישות	תפקיד	מזהה חשוב	הערה אנליטית
gtfs_route	קו/מסלול מתוכנן ליום	operator_ref , line_ref , date	אותה ישות לוגית מקבלת id חדש כל יום (Stride Data Model)
gtfs_ride	נסיעה מתוכננת	id , journey_ref , start_time	בסיס לספירת נסיעות מתוכננות (Stride Data Model)
gtfs_ride_stop	תחנה בתוך נסיעה מתוכננת	gtfs_ride_id , gtfs_stop_id , stop_sequence	בסיס לחישוב איחור בתחנה (GTFS Schedule Reference)
siri_ride	נסיעה בפועל	siri_route_id , journey_ref , vehicle_ref , gtfs_ride_id	הישות החשובה ביותר ל-planned vs actual (Stride DB repository)
siri_vehicle_location	דגימת מיקום רכב	recorded_at_time , lat , lon , velocity	האטום הגיאוגרפי של ביצוע בפועל (Stride Data Model)

ארכיטקטורת Open Bus בקיצור

מאגר open-bus-stride-api ממומש ב-FastAPI ומארגן את ה-routers לפי קבצים, כאשר כל קובץ מייצר prefix כמו /gtfs_routes או (open-bus-stride-api) /siri_rides. דפוס ה-API חוזר כמעט בכל ישות: /<entity>/list לשליפה מרובה עם פילטרים ו- /<entity>/get?id=... לשליפה יחידה (OpenAPI JSON).

מאגר open-bus-pipelines משמש שכבת תזמור Airflow שמפעילה תהליכי SIRI ו-GTFS, snapshots, טעינת GTFS, התאמות בין SIRI ל-GTFS, ועדכון שדות denormalized כמו משך נסיעה ומיקומי רכב ראשוניים/אחרונים (open-bus-pipelines). לפי תיעוד ה-SIRI snapshots, ETL, נמשכים ומעובדים בערך כל 60 שניות, בעוד GTFS נמשך ומעובד במחזור שעות/יומי בהתאם לתהליך (STRIDE_ETL_PROCESSES.md).

מאגר `open-bus-stride-db` מגדיר את סכמת PostgreSQL, מודלי SQLAlchemy ומיגרציות Alembic; הוא לב המיפוי בין טבלאות GTFS ו-SIRI, כולל שדות התאמה כמו, `siri_ride.gtfs_ride_id`, `siri_ride.scheduled_time_gtfs_ride_id` ו-`(open-bus-stride-db)` `siri_ride_stop.gtfs_stop_id`.

כל ה־endpoints במפרט החי

הטבלה הבאה נבנתה ממפרט ה-OpenAPI החי של השירות, ולא רק מהקישור הספציפי ששלחת (OpenAPI JSON). עמודת "פרמטרים שימושיים" לא כוללת פרמטרים גלובליים כמו `limit`, `offset`, `get_count` ו-`order_by`, כדי להבליט את פילטרי הדומיין.

קבוצה	Meth od	Endpoint	פרמטרים חובה	פרמטרים שימושיים
<code>user cases</code>	GET	<code>/rides_execution/list</code>	<code>date_from</code> , <code>date_to</code> , <code>operator_ref</code> , <code>line_ref</code>	<code>date_from</code> , <code>date_to</code> , <code>operator_ref</code> , <code>line_ref</code>
<code>user cases</code>	GET	<code>/route_timetable/list</code>	אין	<code>planned_start_time_date_from</code> , <code>planned_start_time_date_to</code> , <code>line_refs</code>
<code>user cases</code>	GET	<code>/stop_arrivals/list</code>	אין	<code>gtfs_stop_id</code> , <code>gtfs_ride_ids</code>
<code>gtfs</code>	GET	<code>/gtfs_agencies/list</code>	אין	<code>date_from</code> , <code>date_to</code>
<code>gtfs</code>	GET	<code>/gtfs_ride_stops/get</code>	<code>id</code>	<code>id</code>
<code>gtfs</code>	GET	<code>/gtfs_ride_stops/list</code>	אין	<code>...arrival_time_from</code> , <code>arrival_time_to</code> , <code>gtfs_stop_ids</code> , <code>gtfs_ride_ids</code> , <code>gtfs_ride__gtfs_route_id</code> , <code>gtfs_ride__journey_ref_prefix</code> , <code>gtfs_ride__start_time_from</code> , <code>gtfs_ride__start_time_to</code>
<code>gtfs</code>	GET	<code>/gtfs_rides/get</code>	<code>id</code>	<code>id</code>
<code>gtfs</code>	GET	<code>/gtfs_rides/list</code>	אין	<code>...gtfs_route_id</code> , <code>journey_ref_prefix</code> , <code>start_time_from</code> , <code>start_time_to</code> , <code>gtfs_route__date_from</code> , <code>gtfs_route__date_to</code> , <code>gtfs_route__line_refs</code> , <code>gtfs_route__operator_refs</code>
<code>gtfs</code>	GET	<code>/gtfs_routes/get</code>	<code>id</code>	<code>id</code>
<code>gtfs</code>	GET	<code>/gtfs_routes/list</code>	אין	<code>...date_from</code> , <code>date_to</code> , <code>line_refs</code> , <code>operator_refs</code> , <code>route_short_name</code> , <code>route_long_name_contains</code> , <code>route_mkt</code> , <code>route_direction</code>
<code>gtfs</code>	GET	<code>/gtfs_stops/get</code>	<code>id</code>	<code>id</code>
<code>gtfs</code>	GET	<code>/gtfs_stops/list</code>	אין	<code>date_from</code> , <code>date_to</code> , <code>code</code> , <code>city</code>
<code>siri</code>	GET	<code>/siri_ride_stops/get</code>	<code>id</code>	<code>id</code>

קבוצה	Meth od	Endpoint	פרמטרים חובה	פרמטרים שימושיים
siri	GET	/siri Ride Stops/list	אין	siri_stop_ids, siri_ride_ids, siri_vehicle_location__lon__greater_or_equal, siri_vehicle_location__lon__lower_or_equal, siri_vehicle_location__lat__greater_or_equal, ...siri_vehicle_location__lat__lower_or_equal, siri_vehicle_location__recorded_at_time_from, siri_vehicle_location__recorded_at_time_to
siri	GET	/siri Rides/get	id	id
siri	GET	/siri Rides/list	אין	gtfs_route__date_from, gtfs_route__date_to, gtfs_route__line_refs, gtfs_route__operator_refs, gtfs_route__route_short_name, gtfs_route__route_long_name_contains, gtfs_route__route_mkt, ...gtfs_route__route_direction
siri	GET	/siri Routes/get	id	id
siri	GET	/siri Routes/list	אין	line_refs, operator_refs
siri	GET	/siri Snapshots/get	id	id
siri	GET	/siri Snapshots/list	אין	snapshot_id_prefix
siri	GET	/siri Stops/get	id	id
siri	GET	/siri Stops/list	אין	codes
siri	GET	/siri Vehicle Locations/get	id	id
siri	GET	/siri Vehicle Locations/list	אין	...siri_vehicle_location_ids, siri_snapshot_ids, siri_ride_stop_ids, recorded_at_time_from, recorded_at_time_to, lon__greater_or_equal, lon__lower_or_equal, lat__greater_or_equal
siri	GET	/siri Velocity Aggregation/siri Velocity Aggregation	recorded_from	recorded_from, lon_min, lon_max, lat_min, lat_max, rounding_precision
aggregations	GET	/gtfs Rides Agg/group_by	date_from, date_to, group_by	date_from, date_to, exclude_hours_from, exclude_hours_to, group_by
aggregations	GET	/gtfs Rides Agg/list	date_from, date_to	date_from, date_to, exclude_hours_from, exclude_hours_to

פרמטרים גלובליים

נמעט כל endpoints מסוג `list` תומכים ב- `get_count` , `offset` , `limit` ו- `order_by` , כאשר ברירת המחדל התיעודית היא בדרך כלל 100 תוצאות והמסמך מציין אפשרות להגדלה עד 500,000 תוצאות (Open Bus Stride API Docs). עם זאת, ניתן הקוד מצא שה-API אוכף בפועל תקרה קשיחה נמוכה יותר לבקשה רגילה, ולכן בניתוחים גדולים עדיף לעבוד בחלונות זמן קטנים, בעימוד, ובדגימה לפני משיכה מלאה (open-bus-stride-api).

סכמות תגובה עיקריות

Schema	שדות מרכזיים
GtfsAgencyPydanticModel	date , operator_ref , agency_name
GtfsRoutePydanticModel	id , date , line_ref , operator_ref , route_short_name , route_long_name , route_mkt , route_direction , route_alternative , agency_name , route_type
GtfsStopPydanticModel	id , date , code , lat , lon , name , city
GtfsRidePydanticModel	id , gtfs_route_id , journey_ref , start_time , end_time
GtfsRideStopPydanticModel	id , gtfs_stop_id , gtfs_ride_id , arrival_time , departure_time , stop_sequence , pickup_type , drop_off_type , shape_dist_traveled
SiriRoutePydanticModel	id , line_ref , operator_ref
SiriRidePydanticModel	id , siri_route_id , journey_ref , scheduled_start_time , vehicle_ref , updated_first_last_vehicle_locations , first_vehicle_location_id , last_vehicle_location_id , updated_duration_minutes , ... duration_minutes , journey_gtfs_ride_id , route_gtfs_ride_id
SiriStopPydanticModel	id , code
SiriRideStopPydanticModel	id , siri_stop_id , siri_ride_id , order , gtfs_stop_id , nearest_siri_vehicle_location_id
SiriVehicleLocationPydanticModel	id , siri_snapshot_id , siri_ride_stop_id , recorded_at_time , lon , lat , bearing , velocity , distance_from_journey_start , distance_from_siri_ride_stop_meters
SiriSnapshotPydanticModel	id , snapshot_id , etl_status , etl_start_time , etl_end_time , error , num_successful_parse_vehicle_locations , num_failed_parse_vehicle_locations , num_added_siri_rides , num_added_siri_ride_stops , ... num_added_siri_routes , num_added_siri_stops
SiriVelocityAggregationPydanticModel	rounded_lon , rounded_lat , total_sample_count , average_rolling_avg , stddev_rolling_avg
RouteTimetablePydanticModel	id , name , city , lon , lat , planned_arrival_time , gtfs_line_ref , gtfs_line_start_time , gtfs_ride_id
StopArrivalPydanticModel	actual_arrival_time
RideExecutionPydanticModel	planned_start_time , actual_start_time , gtfs_ride_id
GtfsRidesAggPydanticModel	gtfs_route_id , gtfs_route_hour , num_planned_rides , num_actual_rides , operator_ref
GtfsRidesAggGroupByPydanticModel	gtfs_route_date , gtfs_route_hour , operator_ref , day_of_week , line_ref , route_short_name , route_long_name , total_routes , total_planned_rides , total_actual_rides

איך לבחור endpoint לפי שאלה תחבורתית

שאלה	נקודת התחלה	המשך ניתוח	הערת זהירות
אילו קווים קיימים למפעיל/מספר קו?	/gtfs_routes/list	סינון לפי operator_refs , route_short_name , date_from , date_to	להשתמש ב־ line_ref , לא רק במספר הקו לנוסע
מה לוח הזמנים של קו?	/route_timetable/list	line_refs + חלון planned_start_time	endpoint מוגבל ל-use case, לא מחליף שליפה מלאה של gtfs_ride_stops
אילו נסיעות תוכננו?	/gtfs_rides/list	הצטרפות ל־ gtfs_routes דרך gtfs_route_id או פילטרים קשורים	להקפיד על timezone בפילטרי זמן

שאלה	נקודת התחלה	המשך ניתוח	הערת זהירות
מה קרה בפועל בקו?	/siri_rides/list	סינון לפי scheduled_start_time , siri_route_line_refs , siri_route_operator_refs	הוא הערך הראשון שהתקבל, לא בהכרח עדכון אחרון
איפה היה רכב מסוים?	/siri_vehicle_locations/list	סינון לפי siri_ride_vehicle_ref , זמן ומרחב	יש טעות כתיב בפרמטרים מסוימים: scheduled
האם יש בעיית קליטה ב-SIRI?	/siri_snapshots/list	בדיקת etl_status , שגיאות ומספרי parsing	להשתמש רק ב-snapshots במצב loaded לניתוח אמין
מה היחס בין מתוכנן למבוצע?	/rides_execution/list	date_from , date_to , operator_ref , line_ref	השוואה ברמת התחלה בלבד, לא בהכרח כל תחנה
מהירות/עומס גיאוגרפי	/siri_velocity_aggregation/siri_velocity_aggregation	lat/lon ו-recorded_from	אגרגציה מרחבית, לא ניתוח קו ספציפי

דוגמאות חיות שנבדקו

בדיקת /gtfs_agencies/list?limit=5 החזירה דוגמאות מפעילים, כולל operator_ref=3 עבור אגד, ולכן זוהי נקודת פתיחה טובה למציאת מזהי מפעיל לפני ניתוח קווים (בדיקת gtfs_agencies). בדיקת /gtfs_routes/list עבור קו 480 בתאריך 2026-05-17 החזירה line_ref=7020 ו-line_ref=7022 עבור חלופות שונות של הקו, מה שממחיש מדוע אסור להסתפק ב-route_short_name בלבד (בדיקת קו 480). בדיקת /gtfs_stops/list עבור ירושלים בתאריך הבדיקה החזירה תחנות GTFS עם name , lon , lat , code , date , ו-city , ולכן תחנה ב-Stride היא ישות יומית בעולם GTFS ולא רק קוד תחנה קבוע (בדיקת gtfs_stops). בדיקת /siri_snapshots/list?order_by=id desc הראתה snapshot עכשווי במצב loading , ולכן אין להניח שכל snapshot אחרון כבר ראוי לניתוח לפני בדיקת etl_status (בדיקת siri_snapshots).

מתכוני KPI מעשיים

ביצוע נסיעות מול תכנון

הגדרה בסיסית: מספר נסיעות GTFS מתוכננות מול מספר נסיעות SIRI שהותאמו ל-GTFS באותו קו, מפעיל ויום. ברמה פשוטה אפשר להתחיל מ-/rides_execution/list , שמחזיר , planned_start_time , actual_start_time ו-gtfs_ride_id , אך בניתוח מקצועי כדאי לבדוק גם את שיעור ה-NULL ב-gtfs_ride_id ואת מצב ה-snapshots (OpenAPI JSON).

```
import requests

BASE = "https://open-bus-stride-api.hasadna.org.il"
params = {
    "date_from": "2026-05-17",
    "date_to": "2026-05-17",
    "operator_ref": 3,
    "line_ref": 7020,
    "limit": 5000,
}

rows = requests.get(f"{BASE}/rides_execution/list", params=params, timeout=30).json()

planned = sum(1 for r in rows if r["planned_start_time"] is not None)
```

```
actual = sum(1 for r in rows if r["actual_start_time"] is not None)
matched = sum(1 for r in rows if r["gtfs_ride_id"] is not None)
```

איחור בתחנה

החישוב העקרוני הוא `actual_arrival_time - planned_arrival_time`, אבל ב־“actual arrival” Stride אינו תמיד אירוע עצירה מפורש; לעיתים הוא נגזר מהמיקום הקרוב ביותר של הרכב לתחנת GTFS. לכן בניחוח מקצועי צריך לסנן לפי `distance_from_siri_ride_stop_meters`, לבדוק שה־`gtfs_ride_id` אינו `NULL`, ולהפריד בין תחנה ראשונה, תחנות ביניים ותחנה אחרונה ([Stride Data Model](#)).

```
gtfs_ride_stop.arrival_time
מול
siri_ride_stop.nearest_siri_vehicle_location_id
→ siri_vehicle_location.recorded_at_time
```

Headway מתוכנן ומבוצע

Headway מתוכנן נמדד לפי פערי `gtfs_ride.start_time` או לפי `gtfs_ride_stop.arrival_time` בתחנה מסוימת, בעוד Headway בפועל צריך להימדד לפי זמני הגעה בפועל לאותה תחנה או לפי דגימות SIRI סמוכות. מדד זה רגיש מאוד לשגיאות התאמה, ולכן יש להפריד בין קווים תדירים, קווים דלילים וקווים עם חלופות רבות ([דוח KPI הרשות לתחבורה ציבורית](#)).

מהירות מסחרית וקלטות צווארי בקבוק

ניתוח מהירות יכול להתחיל מ־`/siri_vehicle_locations/list` ברמת רכב או מ־`/siri_velocity_aggregation/siri_velocity_aggregation` ברמת grid גיאוגרפי. התוצר צריך להיות מפה או טבלת מקטעים, ולא ממוצע קו יחיד, כי צווארי בקבוק נוצרים בדרך כלל בצמתים, רחובות וקטעים ספציפיים.

GTFS בישראל: דברים שחייבים לזכור

משרד התחבורה מפרסם את נתוני התכנון ב־GTFS, והגרסה העדכנית של תיעוד המפתחים מציינת מקור HTTPS ב־`https://gtfs.mot.gov.il/gtfsfiles/`, עדכון כל לילה, קבצי ZIP, קידוד UTF-8 BOM ומעבר לקבצים כגון `tariff_2022.zip`, `zones_2022.zip`, `RouteNetworksByDate.zip` ו־([GTFS Documentation v3](#)) `ChargingRavKav.zip`. תיעוד ישן יותר עדיין מזכיר FTP וקבצים כמו `ClusterToLine.zip` ו־`TripIdToDate.zip`, ולכן בעת בניית pipeline חדש יש להעדיף את התיעוד העדכני אך להבין שמערכות היסטוריות עשויות להחזיק לוגיקה ישנה ([GTFS Developer Information](#)).

ב־GTFS הישראלי, יום השירות של אוטובוסים ושירותים שאינם רכבת ישראל מוגדר לרוב מ־04:00 עד 03:59 למחרת, ולכן שעות כמו `25:30:00` מתארות 01:30 בלילה של היום הקלנדרי הבא אך עדיין שייכות ליום השירות הקודם ([GTFS Documentation v3](#)). זו נקודת כשל נפוצה: אם מנתחים לפי תאריך קלנדרי במקום יום שירות, נסיעות לילה יעברו יום וייצרו טעויות בביצוע מול תכנון.

SIRI בישראל: מה מגיע ומה לא

ממשק SIRI-SM של משרד התחבורה מספק חיזויי הגעה בזמן אמת לתחנות עבור אוטובוסים ורכבות, כולל `VehicleLocation` עבור אוטובוס שנמצא בנסיעה, והקואורדינטות הן ב־[\(SIRI-SM FAQ\)](#) WGS1984. הממשק המקורי של משרד התחבורה הוא SOAP/XML ודורש `RequestorRef`, רישום וכתובת IP סטטית, אבל Open Bus Stride חוסך את רוב המורכבות הזו למשתמש האנליטי בכך שהוא כבר קולט snapshots ומנגיש אותם כ־JSON דרך REST [\(SIRI-SM FAQ\)](#).

המשמעות החשובה ביותר היא ש־SIRI אינו "אמת מוחלטת" אלא זרם תצפיות. כאשר אין דגימה, כאשר snapshot במצב `loading`, כאשר ה־ETL עדיין לא העשיר `gtfs Ride id`, או כאשר הדגימה הקרובה לתחנה רחוקה מדי פיזית, המדד עלול להיראות כמו איחור או אי־ביצוע אף שבפועל מדובר בפער נתונים.

מקור רשמי למדדי איכות שירות

הרשות הארצית לתחבורה ציבורית משתמשת במסגרת KPI הכוללת נסיעות נוסע, עלות תפעולית, שביעות רצון, אמינות וזמינות, ומציינת מקורות כגון תיקופים במסלקה, מיקומי אוטובוסים בזמן אמת, GTFS, סקרי שביעות רצון וסקרי עמדות ([דוח מדדי ביצוע מרכזיים](#)). החיבור ל־Open Bus הוא בכך ש־Stride מכסה היטב את רכיבי התכנון והביצוע, אך אינו מכסה באופן ציבורי מלא תיקופים, תפוסה, הכנסות או סקרי שביעות רצון.

KPI	אפשרי עם Stride?	מקור עיקרי	מה חסר להשלמה מקצועית
קיום נסיעות	כן	GTFS + SIRI	אימות מול AVL פנימי/דוחות מפעיל
איחור בתחנה	כן, בקירוב	<code>siri_vehicle_location</code> + <code>gtfs_ride_stop</code>	אירוע עצירה ודלתות בפועל
Headway	כן	GTFS + SIRI	זיהוי מדויק של הגעה לתחנה
מהירות מסחרית	כן	SIRI locations	הפרדה בין נסיעה, עצירה, terminal time
זמינות תכנונית	חלקית	GTFS + GIS	אוכלוסייה, שימושי קרקע, רשת הליכה
ביקושים	לא במלואו	תיקופים/מסלקה	נתוני Rav-Kav ואפליקציות ברמת פרט
עלות לנוסע	לא	עלויות תפעול + תיקופים	עלויות ושכר, ק"מ, שעות נהג
שביעות רצון	לא	סקרים	שאלוני נוסעים ומדדי חוויית שירות

טעויות נפוצות ואיך להימנע מהן

- לא להשתמש במספר קו לנוסע כמפתח: `route_short_name=480` יכול להתפצל לכמה `line_ref`, כיוונים וחלופות, ולכן צריך לזהות את `operator_ref + line_ref + date` לפני ניתוח.
- לא לערבב GTFS יומי עם SIRI מצטבר: `gtfs_route` ו־`gtfs_stop` הם יומיים, בעוד `siri_route` ו־`siri_stop` אינם יומיים באותו אופן.
- לא לחשב KPI לפני בדיקת איכות נתונים: בדוק `siri_snapshot.etl_status`, אחוז `gtfs_ride_id IS NULL`, ודגימות במרחק חריג מהתחנה.

- לא להניח שה־`endpoint` האגרגטיבי תמיד עובד: בבדיקה חיה, `/gtfs_rides_agg/group_by` החזיר `permission denied for materialized view`, ולכן צריך fallback לחישוב עצמאי.
- לא להתעלם מ־`timezone` ויום שירות: GTFS ישראלי משתמש ביום שירות 28 שעות, בעוד מסד הנתונים עובד עם timestamps מלאים ו־UTC.

מסלול למידה ושינון

שלב ראשון: מפת מזהים

שנן קודם את הצמדים הבאים: `operator_ref + line_ref` מזהים מסלול לוגי, `gtfs_route.date` קובע את גרסת היום, `gtfs_stop.code` הוא קוד תחנה ציבורי, ו־`siri_ride.gtfs_ride_id` הוא הקיצור החשוב ביותר בין ביצוע לתכנון.

שלב שני: חמש שאילתות חובה

1. מצא מפעיל: `/gtfs_agencies/list`.
2. מצא קו וחלופה: `/gtfs_routes/list`.
3. מצא נסיעות מתוכננות: `/gtfs_rides/list`.
4. מצא נסיעות בפועל: `/siri_rides/list`.
5. בדוק איכות SIRI: `/siri_snapshots/list`.

שלב שלישי: תרגילי עבודה

תרגיל	מטרה	תוצר
קו אחד, יום אחד	להבין <code>planned vs actual</code>	טבלת <code>planned_start</code> מול <code>actual_start</code>
תחנה אחת, קו אחד	למדוד פערי הגעה	התפלגות איחורים לפי שעה
מפעיל אחד, שבוע	לזהות קווים בעייתיים	דירוג לפי אחוז התאמה/אי־ביצוע
אזור גיאוגרפי	לזהות צווארי בקבוק	heatmap מהירויות או זמני נסיעה
השוואת חלופות	להפריד כיוון וחלופה	KPI לפי <code>route_alternative</code> , <code>route_direction</code> , <code>line_ref</code>

צ'קליסט לפני כל ניתוח

- האם בחרת תאריך GTFS נכון ויום שירות נכון?
- האם זיהית `operator_ref` ו-`line_ref`, ולא רק מספר קו?
- האם בדקת שה-snapshots הרלוונטיים במצב `loaded` ?
- האם בדקת את שיעור הנסיעות שבהן `gtfs Ride id` הוא `NULL` ?
- האם סיננת דגימות מיקום רחוקות מדי מתחנה כאשר חישבת `arrival`?
- האם הפרדת בין כיוון, חלופה, חלון שעה וימי שבוע?
- האם שמרת את ה-URL query ואת גרסת התיעוד/תאריך הריצה כדי לשחזר?

נספח: קוד בסיסי למשיכת API ל-DataFrame

```
import requests
import pandas as pd

BASE = "https://open-bus-stride-api.hasadna.org.il"

def get_df(path, **params):
    params.setdefault("limit", 15000)
    r = requests.get(BASE + path, params=params, timeout=60)
    r.raise_for_status()
    return pd.DataFrame(r.json())

routes = get_df(
    "/gtfs_routes/list",
    date_from="2026-05-17",
    date_to="2026-05-17",
    operator_refs="3",
    route_short_name="480",
)

line_refs = routes["line_ref"].unique().tolist()
print(routes[["line_ref", "route_short_name", "route_long_name", "route_direction", "route_alternative"]])
```

Open Bus Stride הוא לא רק API, אלא מודל עבודה שלם לניתוח תחבורה ציבורית: מקורות ממשלתיים, ETL קהילתי, סכמת DB, endpoints, וכלים אנליטיים. אם משננים את ההבדל בין GTFS לבין SIRI, את המפתחות `gtfs Ride id`, `code`, `date`, `line_ref`, `operator_ref`, ואת מגבלות האיכות של snapshots והתאמות, אפשר לבנות עליו כלי ניתוח חזקים מאוד לתכנון, בקרה, אמינות, ביצוע מול תכנון, ומהירות מסחרית.